



Fast density peaks clustering algorithm based on improved mutual K-nearest-neighbor and sub-cluster merging

Chao Li ^{a,b}, Shifei Ding ^{a,b,*}, Xiao Xu ^{a,b}, Haiwei Hou ^{a,b}, Ling Ding ^c

^a School of Computer Science and Technology, China University of Mining and Technology, Xuzhou 221116, China

^b Mine Digitization Engineering Research Center of Ministry of Education of the People's Republic of China, Xuzhou 221116, China

^c College of Intelligence and Computing, Tianjin University, Tianjin 300350, China

ARTICLE INFO

Keywords:

Density peaks clustering
Improved mutual K-nearest-neighbor
Kd-tree
Sub-cluster merging
Sparse matrix

ABSTRACT

Density peaks clustering (DPC) has had an impact in many fields, as it can quickly select centers and effectively process complex data. However, it also has low operational efficiency and a “Domino” effect. To solve these defects, we propose a fast density peaks clustering algorithm based on improved mutual K-nearest-neighbor and sub-cluster merging (KS-FDPC). The new algorithm adopts a partitioning-merging strategy. By dividing the data into multiple sub-clusters, the impact range of high-density points on subsequent allocation points can be reduced. And the fast nearest neighbors search improves the speed of DPC. In the experiment, KS-FDPC is used to compare with eight improved DPC algorithms on eight synthetic data and eight UCI data. The results indicate that the overall clustering performance of KS-FDPC is superior to other algorithms. Moreover, KS-FDPC runs faster than other algorithms. Therefore, KS-FDPC is an effective improvement of DPC.

1. Introduction

As an inter-discipline in machine learning and database, data mining can search for hidden knowledge from massive data to support decision-making. As a preprocessing method, clustering makes a difference in data mining. By grouping different data, clustering aims to maximize intra-class affinity and minimize inter-class affinity. Each class shows some overall characteristics of data distribution so that one or more of them can be efficiently processed under the condition of limited resources. Clustering analysis has extensive applications, including text mining [1], image retrieval [2], social networking [3], and e-commerce shopping [4]. Recently a mass of excellent clustering algorithms has been put forward [5–9], among which partition-based [5–7] and density-based clustering methods [8,9] are the main research directions of clustering algorithms. K-means [10] and GMM [11] are two classical partition-based clustering algorithms with fast convergence speed and simple implementation. However, the density-based clustering algorithms [12,13] have more advantages when dealing with data with complex shapes. But this kind of algorithm often needs to calculate the similarity between arbitrary sample pairs, which has high time complexity and can not efficiently process a large number of data.

Density peaks clustering (DPC) [14] is one of the density-based clustering algorithms. It uses the similarity matrix to estimate the local density of each sample and then combines the relative distance to select center points and complete the clustering. The algorithm is apt to program and can effectively deal with many non-convex datasets. However, the higher complexity ($O(n^2)$) required by DPC will limit its application on a large amount of data [17,18]. What's more, DPC will produce a “Domino” effect when assigning non-

* Corresponding author at: School of Computer Science and Technology, China University of Mining and Technology, Xuzhou 221116, China.
E-mail address: dingsf@cumt.edu.cn (S. Ding).

central points [20], that is, the wrong allocation of a sample will produce a chain reaction.

Aiming at the clustering efficiency and “Domino” effect of DPC, we propose a fast density peaks clustering based on improved mutual K-nearest neighbors and sub-cluster merging (KS-FDPC). KS-FDPC takes far less time to process large-scale data than other improved DPC algorithms. In KS-FDPC, we have mainly made the following three contributions:

- 1) The improved mutual KNN is adopted to divide the dataset and calculate the sample density. For the relative distance of each sample point, we only search it within the mutual KNN, so the time complexity of the algorithm can be effectively reduced to $O(n \log n)$.
- 2) After generating reams of initial sub-clusters, the computed inter-cluster similarities are sorted from large to small. The two sub-clusters with maximum similarity are merged in turn. The algorithm stops until the number of sub-clusters generated equals that of real sub-clusters, or the maximum similarity between sub-clusters is zero.
- 3) We replace the fully connected matrix in KS-FDPC with a sparse matrix to cut down the space resource and deal with large-scale datasets.

In this section, we briefly introduce the DPC algorithm and its shortcomings and propose a new algorithm. The research on the DPC algorithm and the details of the new algorithm will be introduced in subsequent sections.

The structure of the remaining part is as follows. In Section 2 we introduce the DPC algorithm and analyze its shortcomings. In Section 3, the idea and steps of KS-FDPC are described in detail. We present and analyze the experimental results obtained by all algorithms in Section 4. Finally, the full text and the next work are summarized.

2. Related work

For sake of clarity, two tables, Table 1 and Table 2, are provided to describe the abbreviations and symbols of the entire text, respectively.

2.1. DPC algorithm

DPC [14] has received widespread attention since its inception. In DPC, the relationship between all peak points and other points is analyzed in detail, and two measures of density peak points are put forward. 1) Local density ρ_i , and 2) relative distance δ_i . In DPC, points with higher local densities and larger relative distances are more likely to be selected as peak points. The formulas for them are (1) and (3), respectively.

Given the data set $X = \{x_1, x_2, \dots, x_n\} \in \mathbb{R}^{n \times m}$, then

$$\rho_i = \sum_{j=1}^n sim_{ij} \quad (1)$$

Table 1
Abbreviations and their descriptions.

Abbreviation	Description
DPC	Density peaks clustering [14]
KS-FDPC	Fast density peaks clustering algorithm based on improved mutual K-nearest neighbor and sub-cluster merging
GMM	Gaussian mixed model [6]
KNN	K-nearest neighbors [37]
RKNN	Reverse K-nearest neighbors [37]
SNN	Shared nearest neighbors [17]
MKNN	Mutual K-nearest neighbors [22]
IMKNN	Improved mutual K-nearest neighbors [22]
SNN-DPC	Shared-nearest-neighbor-based clustering by fast search and find of density peaks [17]
NN	Natural nearest neighbors [31]
FastDP	Fast and general density peaks clustering [33]
LDP-MST	Clustering with local density peaks-based minimum spanning tree [31]
Fast LDP-MST	An efficient density-peak-based clustering method for large-size datasets [30]
RP-DIV	Random pair division [30]
DPCSA	Density peaks clustering based on weighted local density sequence and nearest neighbor assignment [24]
DPC-KNN-PCA	Study on density peaks clustering based on k-nearest neighbors and principal component analysis [23]
SDPC	An improved density peaks clustering algorithm with fast finding cluster centers [25]
FSDPC	A fast density peaks clustering algorithm with sparse search [26]
PS-DPC	A variant of DPC [27]
ACC	The accuracy rate [28]
ARI	The adjusted rand index [29]
AMI	The adjusted mutual information [29]
RT	Running time
KS-FDPC-Full	KS-FDPC with brute force nearest-neighbor search
KS-FDPC-sparse	KS-FDPC with sparse matrix

Table 2
Symbols and their descriptions.

Symbol	Description
n	The number of sample points in the dataset
m	The number of attributes in each sample
$X = \{x_1, \dots, x_n\}$	The dataset with n points
K	The number of nearest neighbors
p	The parameter of DPC
ρ_i	The local density of x_i in DPC
δ_i	The relative distance of x_i in DPC
γ_i	The decision value in DPC
sim_{ij}	The similarity between i -th and j -th samples
d_{ij}	The Euclidean distance between i -th and j -th samples
$\mathbf{M}$	The Euclidean distance matrix in $\mathbf{X}$
M_{sort}	An ascending set of distances between any sample points in $\mathbf{X}$
$DS_knn(i)$	The reverse 2K nearest neighbor set of x_i with respect to Euclidean distance
$dislevel_i$	The distance level of x_i
$IM_knn(i)$	The improved mutual K-nearest neighbor set of x_i
ρ'_i	The newly defined local density of x_i in KS-FDPC
δ'_i	The newly defined relative distance of x_i in KS-FDPC
CON_{ij}	The concentration between i -th and j -th sub-clusters
$Pavg_{ij}$	The average density of points between i -th and j -th sub-clusters
STD_{ij}	The density standard deviation of points after merging i -th and j -th sub-clusters
SIM_{ij}	The similarity between i -th and j -th sub-clusters calculated by (13)
SIM'_{ij}	The similarity between i -th and j -th sub-clusters calculated by (14)

where sim_{ij} represents the similarity measurement between i -th and j -th points. In DPC, different similarity measurement functions are set for datasets with different sizes, which are indicated in formula (2).

$$sim_{ij} = \begin{cases} \chi(d_{ij} - d_c), & \text{large-scale} \\ e^{-((d_{ij}/d_c)^2)}, & \text{small-scale} \end{cases} \quad (2)$$

where the truncation distance d_c is the only parameter of DPC. In DPC, $d_c = M_{sort}(\text{round}(p \cdot n))$. M_{sort} represents an ordered set of all values in the distance matrix $\mathbf{M}$ from small to large. The value of p is about 1% to 2%. When $x < 0$, $\chi(x) = 1$; otherwise $\chi(x) = 0$. It can be seen from formula (2) that when DPC processes data sets with different sizes, the similarity measurement is not uniform, but how to determine the data scale based on the sample size is not mentioned in DPC.

$$\delta_i = \begin{cases} \min_{\exists j: \rho_j > \rho_i} d_{ij} \\ \max_{\forall j: \rho_j < \rho_i} d_{ij} \end{cases} \quad (3)$$

There are two common approaches to selecting centers according to the local density and relative distance calculated in DPC. The first method is manual selection. That is, after drawing a decision graph of ρ and δ , DPC needs a user to manually select the points with larger density and distance as center points. These points are located at the upper right of the graph. However, the artificially selected method will affect the reproducibility of the experiment, and the overlapping points in the decision graph also cause difficulties in selecting peaks. The other is that choose these points correlating with the C largest decision values as the centers. The decision value γ is shown in equation (4). C means the actual number of sub-clusters.

$$\gamma_i = \rho_i \times \delta_i \quad (4)$$

Eventually, the remaining non-center points are allocated.

2.2. Research on DPC algorithm

High time complexity. Many variants of DPC have been proposed to address this issue. The grid method [15,16,32] can classify points with short distances or high similarity into a grid, greatly reducing the number of samples to be processed. But grid methods often fail due to the growth of data dimensions. For a dataset with 15 dimensions, if each dimension is divided into only two grids, the generated grid space will have a total of 2^{15} grid points, resulting in a large number of grids with only one or no sample point. Thus, the number of grid points to be processed is equivalent to the number of samples. Nearest-neighbor search is also an effective method to improve the efficiency of DPC. In DPC, the calculation of local density and relative distance requires a fully connected distance matrix. Due to the small contribution of distant points during calculating similarity, the nearest-neighbor search can avoid unnecessary distance calculations. Sieranoja et al. [33] used approximate KNN to accelerate the nearest neighbor search process and proposed the FastDP algorithm. When calculating the relative distance of the local peak, neighboring points with lower local density will cause a global search for this peak. FastDP has limited clustering efficiency improvement due to ignoring this point. Cheng et al. [31] used Kd-

tree to accelerate the nearest-neighbor search and considered each local peak as the initial center while dividing sub-clusters. When calculating the minimum spanning tree between initial sub-clusters, LDP-MST calculates the distance between all sub-clusters. Qiu et al. [30] believe that the distance calculation between sub-clusters of LDP-MST generates lots of redundancy, and as the data size increases, the number of sub-clusters increases sharply. In the worst case, the number of sub-clusters approaches the number of samples, so the distance calculation between sub-clusters urgently needs improvement. Fast LDP-MST only handles sub-clusters with intersections between them and uses Random Pair Division (RP-DIV) to improve the situation where Kd-tree is not suitable for high-dimensional data. To search for nearest-neighbor points, Lu et al. [34] used the Z-value index to map multidimensional data points to a one-dimensional space, which can largely maintain the similarity of the original data. In addition to the grid method and nearest-neighbor search, data sampling can also be used to accelerate the calculation process of DPC. Ding et al. [12] reduced the time complexity of DPC to linear by using mixed sampling based on random and K-means.

“Domino” effect. There are three common strategies to address this issue. One strategy is a two-stage allocation method. This method usually allocates samples with high similarity to the center points after selecting the centers and then allocates the remaining points based on probability. Xie et al. [19] effectively improved the shortcomings of DPC through fuzzy-weighted KNN and two-stage allocation. Tobin et al. [39] used density cores instead of centers to determine the distribution of real sub-clusters. Although the DPCSA proposed by Yu et al. [24] also uses a two-stage method to allocate non-center points, the selection of center points in this algorithm largely depends on human factors. As shown in Fig. 1, (a) and (b) are two decision graphs of DPCSA and represent two different clustering center selections, while (c) and (d) represent the results generated by (a) and (b), respectively. The blue asterisk represents the selected center points, and points with the same color belong to the same sub-cluster in (c) and (d). Different methods of selecting cluster centers result in significant differences in the final results. Another strategy is the partition-merge strategy [21], which first divides the entire dataset into several sub-clusters, and then gradually merges the high-similarity sub-clusters. LDP-MST [31] and Fast LDP-MST [30] adopt this method, which can simultaneously improve clustering accuracy and operational efficiency. The last approach is a hierarchical strategy, which generates sub-clusters within each density level by identifying density levels with significant gaps. The corresponding literature includes [35,36].

The newly proposed algorithm mainly considers the following two aspects. On the one hand, the time complexity of the DPC algorithm mainly depends on the calculation of local density and relative distance, both of which are $O(n^2)$. The calculation of local density can be reduced to K-nearest neighbors, which means that the local density of the sample is determined by the similarity of the nearest neighbor points. Therefore, the calculation of local density can be achieved through a fast nearest neighbor search, and the time complexity can be reduced to $O(n \log n)$. For non-local peaks, their nearest high-density point can be found within the nearest

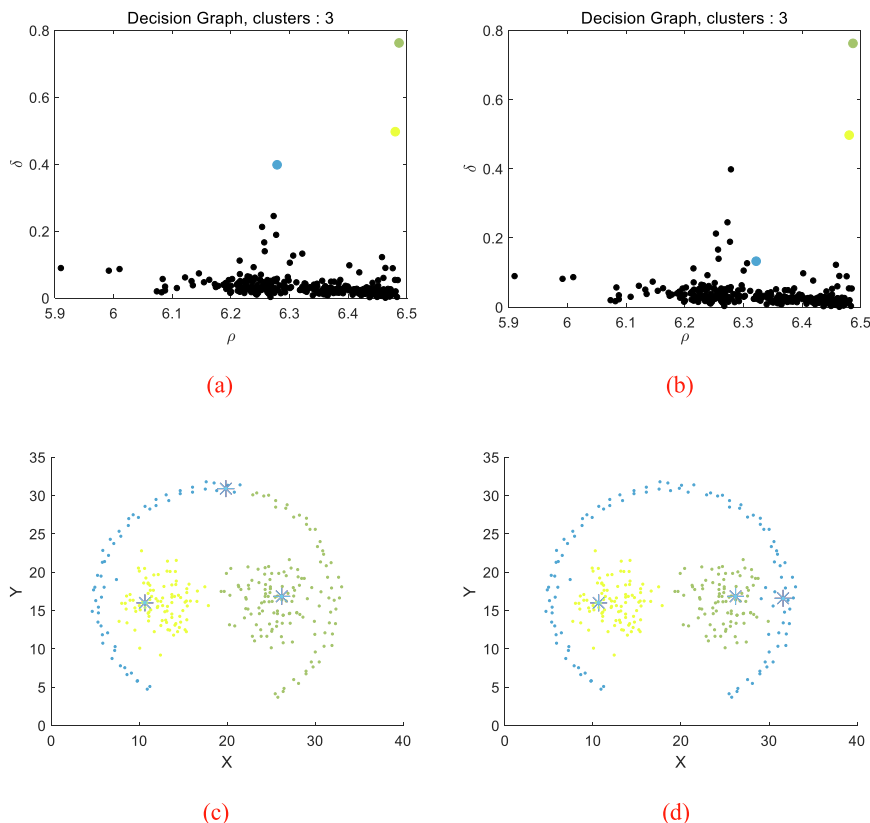


Fig. 1. Clustering results of DPCSA on the Pathbased dataset.

neighbors. For local peaks, the nearest high-density point needs to be globally searched. Therefore, using local peaks as initial center points can reduce the calculation of relative distances between all points. In short, the key to reducing the time complexity of the DPC algorithm lies in fast nearest neighbors search and initial sub-cluster partitioning. Then, merge the initially partitioned sub-clusters to obtain the final result. This is consistent with the partition-merge strategy mentioned above. On the other hand, we adopt a partition-merge strategy to improve DPC while incorporating improved mutual K-nearest neighbors. It places more emphasis on density similarity between samples during the process of partitioning and merging, meaning that points with similar densities will be prioritized and grouped.

The biggest difference between the newly proposed algorithm and others is that it considers improved mutual K-nearest neighbors between samples and reduces the search range of mutual K-nearest neighbors (such as Formula (7) and Algorithm 1 in section 3.3). In the improved mutual K-nearest neighbors [22], the author needs to query all elements of the Euclidean distance matrix. This will undoubtedly increase the time cost of the algorithm. In the newly proposed algorithm, the query range is reduced to 2 K-nearest neighbors. This greatly reduces the running time while maintaining clustering accuracy. Please refer to section 4.4 for the validation of clustering accuracy.

In this section, we provide a detailed introduction to the DPC algorithm, as well as its related research and improvement strategies. At the same time, we provide the research motivation and the differences between the new algorithm and others. A detailed introduction to the new algorithm will be provided in the next section.

3. KS-FDPC algorithm

KS-FDPC mainly consists of three parts, including initial sub-cluster partitioning, initial sub-cluster merging, and final sub-cluster merging. Next, we will introduce these three parts one by one.

3.1. Initial sub-cluster partition

The K-nearest neighbors' method and its variants are often utilized to improve the local density calculation, which includes K-nearest neighbors (KNN), reverse K-nearest neighbors (RKNN), mutual K-nearest neighbors (MKNN) and natural nearest neighbors (NN). As a new KNN variant, the improved mutual K-nearest neighbors (IMKNN) [22] method is proposed by Fan et al. Compared with the MKNN method, the IMKNN method can more effectively divide the sub-clusters with the non-uniform distribution. A new local density of points is redefined in conformity with the IMKNN graph.

At first, through the data $\mathbf{X}$ and parameter K , the Kd-tree is used to obtain the 2KNN of every sample. Based on the idea of RKNN, the IMKNN calculates the number and distance of RKNN, as shown in the formula (5).

$$DS_knn(i) = \{j \mid i \in 2KNN_j\} \quad (5)$$

Compare the number of the reverse nearest neighbors with the K value, and set different distance levels for different points, as shown in formula (6).

$$dislevel_i = \begin{cases} \max_{j \in DS_knn(i)} d_{ij}, 0 < |DS_knn(i)| < K \\ \max_{j \in DS_knn(i) \cap KNN(j)} d_{ij}, |DS_knn(i)| \geq K \end{cases} \quad (6)$$

where $DS_knn(i) \cap KNN(j)$ represents the k -th nearest neighbor to point $\mathbf{x}_i$ in $DS_knn(i)$.

For any sample pair in the IMKNN, if the two points are neighbors within the distance level of each other, the Euclidean distance between them is calculated. Otherwise, the direct distance between them is set to infinity.

$$IM_knn(i) = \{j \mid d_{ij} < \min(dislevel_i, dislevel_j)\} \quad (7)$$

From the formula (7), we can prune the search range of each sample to 2 K nearest neighbors to avoid judging any sample pair in the IMKNN graph.

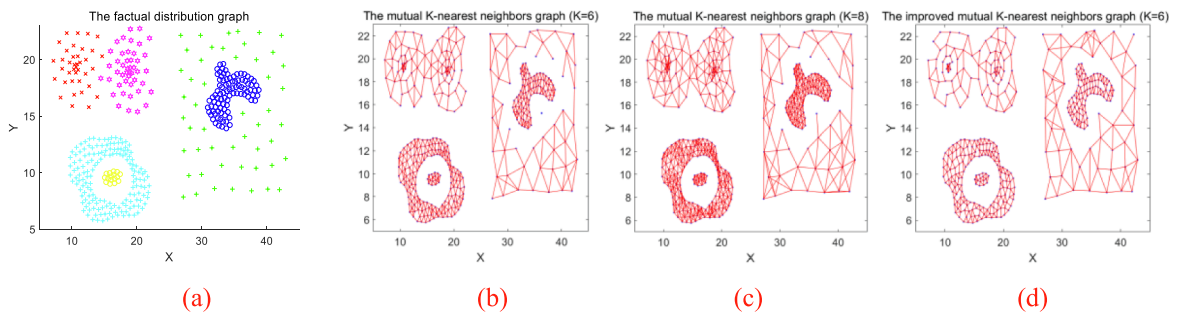


Fig. 2. Nearest neighbors graphs on the Compound dataset.

In the process of constructing a nearest neighbors graph, an intuitive requirement is to connect the points within the same sub-cluster as much as possible, while the number of edges connected between different sub-clusters is relatively small. By comparing the connected graph of the mutual K-nearest neighbors and the improved mutual K-nearest neighbors on the Compound dataset in Fig. 2, it can be seen that the improved mutual K-nearest neighbors method is easier to meet the above requirements. Fig. 2 (a) shows the actual data distribution, with points in different sub-clusters marked with different colors and shapes. (b) and (c) are mutual K-nearest neighbors graphs constructed at $K = 6$ and 8 , respectively. In (b), the number of edges connected between the two upper left sub-clusters is relatively small, making it easier to divide these two sub-clusters. However, in the low-density sub-cluster on the right, there is an outlier that is not connected to any point. This is because the outlier is close to the high-density sub-cluster. In (c), by increasing the number of nearest neighbors K , the outlier in the right low-density sub-cluster can be eliminated, but the connectivity (the number of connected edges) between the two sub-clusters in the upper left corner also further increases, which increases the difficulty in distinguishing these two sub-clusters. Fig. 2 (d) is a neighbor graph constructed using improved mutual K-nearest neighbors. It can be seen from (d) that the improved mutual K-nearest neighbors method not only reduces the number of connecting edges between the two sub-clusters in the upper left corner but also eliminates the outlier of the low-density sub-cluster on the right. Therefore, the improved mutual K-nearest neighbors have certain advantages in partitioning datasets with density levels.

According to the IMKNN of each point in formula (7), the ρ_i and δ_i are recalculated in KS-FDPC, as shown in formulas (8) and (9) respectively.

$$\rho'_i = \frac{|IM_knn(i)|^{\frac{3}{2}}}{\sum_{j \in IM_knn(i)} d_{ij}} \quad (8)$$

$$\delta'_i = \min_{\exists \rho'_j > \rho'_i, j \in IM_knn(i)} d_{ij} \quad (9)$$

In (9), if there is no point with higher density in the IMKNN of the sample point, the sample is taken as the local center. Many initial sub-clusters can be formed by assigning non-central points.

3.2. Initial sub-cluster merging

Many sub-clusters of the initial partition need to be merged in turn according to the level of similarity between them. In KS-FDPC, we define three similarity evaluation indexes among sub-clusters: concentration (*CON*), average density (*Pavg*), and density standard deviation (*STD*). These three indexes are aimed at sub-clusters with IMKNN intersection, that is, some points in one sub-cluster are IMKNN of some points in the other sub-cluster.

Definition 1 (Concentration). Based on the IMKNN described in section 3.1, we calculate the concentration for sub-cluster pairs with intersections. The specific formula is (10).

$$CON_{ij} = \frac{\sum_{p \in C_i, q \in C_j} \frac{1}{d_{pq}}}{\sqrt{M}}, p \in IM_knn(q) \wedge q \in IM_knn(p) \quad (10)$$

where C_i and C_j represent the i -th and j -th sub-clusters respectively. M means the number of non-zero numerators in (10).

Definition 2 (Average density) [21]. When merging sub-clusters, if the average density of two sub-clusters is similar, the two sub-clusters should have a higher similarity. The calculation of average density is seen in (11).

$$Pavg_{ij} = \frac{2\sqrt{Pavg_i \times Pavg_j}}{Pavg_i + Pavg_j} \quad (11)$$

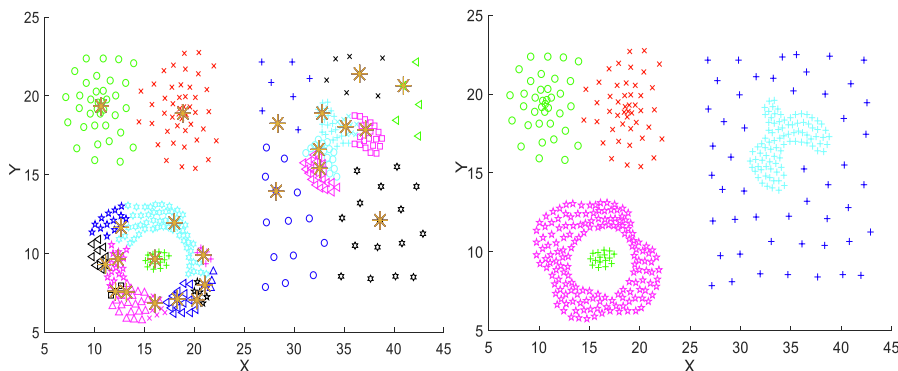


Fig. 3. Running results of KS-FDPC on the Compound dataset.

where $Pavg_i$ represents the average density of all points in the i -th sub-cluster.

Definition 3 (Density standard deviation). If two sub-clusters are merged, the smaller the fluctuation of local density is, the higher the similarity between the two sub-clusters is.

$$STD_{ij} = \frac{Pavg_{ij}}{Std_{ij}} \quad (12)$$

where Std_{ij} is the density standard deviation of all points after the combination of sub-clusters C_i and C_j .

According to the idea of normalization of evaluation indexes proposed by Chen et al. [21], we divide CON by the maximum of all values, while performing the same operation for STD . From the three similarity evaluation indexes defined by formulas (10), (11), and (12), the total similarity evaluation indicator is defined (see formula (13)).

$$SIM_{ij} = CON_{ij} \times Pavg_{ij} + Pavg_{ij} \times STD_{ij} + STD_{ij} \times CON_{ij} \quad (13)$$

Fig. 3 shows the clustering results of KS-FDPC on the Compound dataset, where the left and right subgraphs represent the initially divided sub-clusters and the combined sub-clusters respectively. Points with the same color and style belong to the same sub-cluster, and the asterisk represents the selected center. It can be seen from the figure that KS-FDPC can effectively handle sub-clusters with different densities.

3.3. Final sub-cluster merging

When KS-FDPC merges sub-clusters, the final number of sub-clusters is closely related to the parameter K . If the K value is too small, the number of sub-clusters in formula (13) will exceed the given number of real sub-clusters C . For solving the problem, we propose the second sub-cluster merging strategy as a supplement.

Assume that the sub-cluster set generated after the initial merge is $SC = \{SC_1, \dots, SC_G\}$. On the basis of the number of points in different sub-clusters, SC is sorted from large to small. Among them, the first C sub-clusters are reserved, and the $(C + 1)$ -th, ..., G -th sub-clusters are merged into the sub-clusters with the highest similarity among G sub-clusters. The new similarity formula is shown in (14). In (14), the merged two sub-clusters are required to have the intersection of 2 K -nearest neighbors, that is, part of the points in one sub-cluster are the 2 K -nearest neighbors of part of the samples in the other sub-cluster.

$$SIM'_{ij} = \frac{\min(Pavg_i, Pavg_j)}{\max(Pavg_i, Pavg_j)} \times \frac{\min(Std_i, Std_j)}{\max(Std_i, Std_j)} \times \frac{1}{Std_{ij}} \quad (14)$$

Through formula (14), the closer the average densities of different sub-clusters are, the more similar they are. The closer the density standard deviations of different sub-clusters are, the more similar they are. The smaller the density standard deviation of the merged sub-clusters is, the more similar they are.

Algorithm 1 KS-FDPC

Input: dataset X

Parameter: number of clusters C , number of K -nearest neighbors

Output: clustering result SC

(1) Calculate 2 K -nearest neighbors of every point by Kd-tree.

(2) Based on formula (7), the IMKNN is calculated and saved in the full connection matrix **distM**. The other elements in the matrix are set to infinity.

(3) Calculate the improved local density and relative distance by formulas (8) and (9), and divide sub-clusters.

(4) Calculate the similarity between initial sub-clusters by formula (13). The sub-clusters are merged according to the order of similarity from high to low until the number of sub-clusters is reduced to C or the maximum similarity is zero.

(5) After step 4, if the number of sub-clusters is still higher than C , merge the remaining sub-clusters again according to formula (14).

(6) **return** SC .

3.4. KS-FDPC algorithm

The steps of KS-FDPC are shown in Algorithm 1. From Algorithm 1, the time consumption in step 1 is mainly the nearest neighbors query of Kd-tree, and the time complexity is $O(n \log n)$. To avoid the query of all sample pairs, we only judge the distance from each sample to its 2 K -nearest neighbors when calculating the IMKNN, so the time complexity of step 2 is $O(2nK)$. In step 3, the improved local density and relative distance can be computed in 2 K -nearest neighbors, so the time complexity of step 3 is $O(2nK)$. Suppose the number of sub-clusters in steps 4 and 5 are G_1^2 and G_2^2 respectively, then the complexities of them are $O(G_1^2)$ and $O(G_2^2)$ respectively, and $G_2 < G_1 \ll n$. In addition, from the description of formulas (13) and (14), the similarity of all sub-cluster pairs should not be calculated, only the sub-clusters with the nearest neighbor intersection. Therefore, the actual running time of steps 4 and 5 should be less than $O(G_1^2)$ and $O(G_2^2)$ respectively. Generally, the time complexity of KS-FDPC is $O(n \log n + 2 \times 2nK + G_1^2 + G_2^2) \sim O(n \log n)$, which is significantly less than that of DPC.

Finally, in view of the problem that the fully connected matrix built in step 2 needs a lot of storage resources, we use the sparse matrix method to replace it, that is, the reciprocal of the distance is saved. As a result, the infinite items in the fully connected matrix

become zero, and zero elements are no longer stored in the sparse matrix.

Through the above analysis, the proposed algorithm has certain advantages. Next, we will verify its advantages through experiments.

4. Experiments

For verifying the running rate of KS-FDPC and its improvement on the “Domino” effect on non-convex datasets, we conduct experiments on artificial and UCI datasets in this section. In addition to DPC [14], we also compare KS-FDPC with DPC-KNN-PCA [23], DPCSA [24], SDPC [25], FSDPC [26], PS-DPC [27], LDP-MST [31] and HC-LCCV [38] algorithms. DPC-KNN-PCA utilizes both data dimension reduction and density optimization, wherein only parameter K has a great influence on the results. DPCSA also applies KNN to ameliorate the clustering effect of DPC, but it sets the K to a fixed value of 5. SDPC and FSDPC are two algorithms to reduce the running time of DPC, and both need the truncation distance as the input parameter. In addition, SDPC also needs to set a filtering coefficient to determine the number of filtered points. Based on the provisions of Xu et al., for small-scale data, the filter coefficient of SDPC is set to 0.3. Otherwise, it is set to 0.7. PS-DPC introduces the fuzzy idea into DPC to divide the boundary points more accurately. PS-DPC contains d_c and σ parameters. In this experiment, the value ranges of d_c and σ are set to be [1.0, 2.0] and [0.1, 20] respectively, with steps of 0.1. The optimal result screened by the grid method is the final result. LDP-MST greatly improves the running speed of DPC and does not require any other input parameters except for the actual number of sub-clusters. HC-LCCV can automatically detect the number of sub-clusters and is a completely parameterless algorithm.

Three clustering evaluation indexes are used to measure the pros and cons of different algorithms, including the accuracy rate (ACC) [28], the adjusted Rand index (ARI) [29], and the adjusted mutual information (AMI) [29]. The larger the value of each index is, the better the experimental result of the clustering algorithm is. Meanwhile, the average of 10 running times (RT) of each algorithm is taken to measure the efficiency of the algorithm.

The experimental environment includes Windows 10 operating system, core i7 processor, 8 GB RAM, and MATLAB 2016b.

The details of the synthetic and UCI datasets used in the experiment are shown in Tables 3 and 4, respectively. In addition to the name of the dataset, Tables 3 and 4 also describe the size, amount of attributes, and the actual amount of sub-clusters of each dataset in detail.

4.1. Results and analysis on synthetic datasets

Eight composite datasets are shown in Table 3. The dimensions of the synthesized data are all 2-dimensional and any sample size is no more than 1000, including datasets with density differences between clusters and irregular flow patterns. Experiments on synthetic datasets are mainly used to verify whether KS-FDPC can effectively alleviate the “Domino” effect. The following are the experimental effect diagrams of all control algorithms on the Pathbased, Compound, Jain, circledata, and D13 datasets. Different sub-clusters are plotted with different colors, and the asterisk indicates the center of the sub-cluster. In LDP-MST, HC-LCCV, and KS-FDPC, the number of center points selected far exceeds the actual amount of sub-clusters, so they will not be labeled.

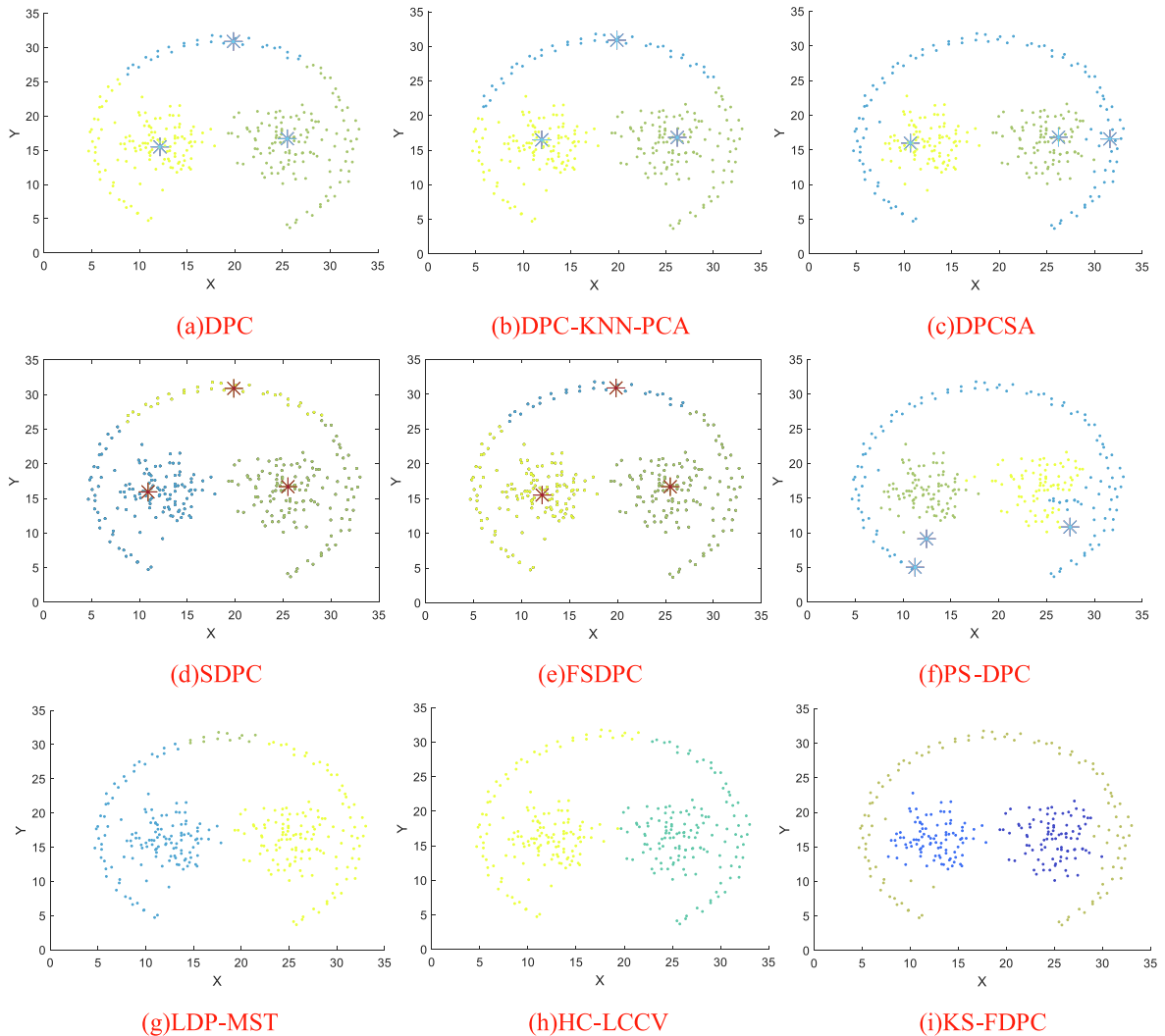
The datasets in Figs. 4 and 5 both have unbalanced density distribution. On such data, DPC preferentially assigns high-density points and divides low-density points into high-density sub-clusters, thus producing a “Domino” effect. DPC-KNN-PCA adopts KNN to improve the density computation and can identify some low-density sub-clusters. But it can’t effectively avert the “Domino” effect because the algorithm still uses the assignment method of DPC. On Pathbased, DPCSA obtains exactly correct clustering results. During the experiment, the selection of center points of the DPCSA algorithm is determined by the user. For low-density cluster centers, some points with low-density are often needed to be chosen. This is difficult for users. SDPC and FSDPC are two improvements to increase the efficiency of DPC. Therefore, they inevitably have a “Domino” effect. PS-DPC uses fuzzy ideas to accurately divide the boundary points of different clusters, but it is similar to the DPC-KNN-PCA, which is an improvement of local density calculation and still assigns remaining points in the same way as DPC. LDP-MST first constructs a minimum spanning tree between sub-clusters and then generates connected components by deleting the edge with the longest distance. Each connected component serves as a sub-cluster. Therefore, on datasets with density levels, LDP-MST prioritizes the removal of edges between low-density sub-clusters. This will result in low-density sub-clusters preferentially merging into their adjacent high-density sub-clusters, while high-density sub-clusters cannot be effectively divided. For example, on Pathbased, LDP-MST allocates many low-density points to high-density sub-clusters. On Compound, LDP-MST divides the low-density sub-cluster on the right into three sub-clusters, while the two sub-clusters in the upper left corner are

Table 3
Synthetic datasets.

Datasets	Sizes	Attributes	Clusters
Pathbased	300	2	3
Compound	399	2	6
Aggregation	788	2	7
Spiral	312	2	3
flame	240	2	2
Jain	373	2	2
circledata	300	2	2
D13	588	2	13

Table 4
UCI datasets.

Datasets	Sizes	Attributes	Clusters
Iris	150	4	3
Wine	178	13	3
Seeds	210	7	3
Segmentation	2310	19	7
Waveform	5000	21	3
Pendigits	10,992	16	10
Gamma	19,020	10	2
Letter	20,000	16	26

**Fig. 4.** Clustering results on the Pathbased dataset.

merged. HC-LCCV automatically generates two sub-clusters on Pathbased, resulting in the loss of a low-density sub-cluster. On Compound, it selects five sub-clusters and mistakenly identifies the low-density and high-density sub-clusters on the right as one. KS-FDPC uses the IMKNN to assign the adjacent sample points with distinct density differences to different sub-clusters effectively. Moreover, the algorithm utilizes multiple sub-clusters to reduce the size of each sub-cluster. With the increase of the sub-clusters, the number of subsequent samples affected by the misallocation of one point will decrease, so the “Domino” effect can be further suppressed.

Figs. 6, 7, and 8 show three datasets with flow pattern structure. On Jain, DPC-KNN-PCA, PS-DPC, LDP-MST, and KS-FDPC

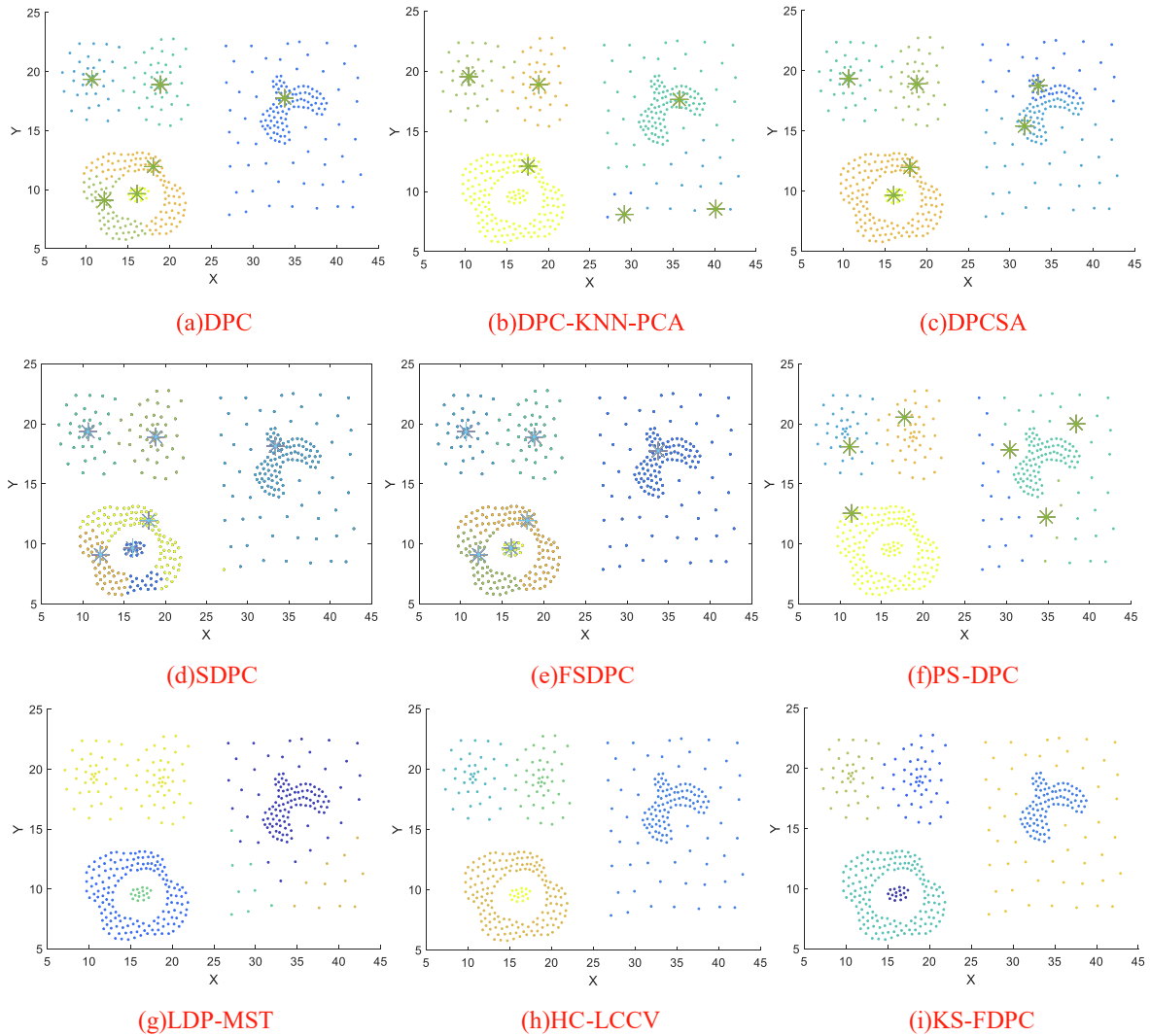


Fig. 5. Clustering results on the Compound dataset.

accurately partition different sub-clusters, and yet the centers identified by DPC-KNN-PCA and PS-DPC lie in the edge region. Other algorithms all select multiple centers in high-density sub-clusters, so low-density sub-clusters are misclassified into high-density sub-clusters. The circledata is composed of two ring sub-clusters, in which the density of the inner ring sub-cluster is higher than that of the outer ring sub-cluster, and each sub-cluster is divided into high-density regions and low-density regions. DPC, DPC-KNN-PCA, DPCSA, SDPC, and FSDPC choose two points of the inner ring sub-cluster as the centers, so there are errors in the selection of clustering centers. HC-LCCV automatically recognizes and divides two sub-clusters into four sub-clusters. PS-DPC, LDP-MST, and KS-FDPC correctly partition the different sub-clusters of the circledata dataset. But PS-DPC tends to select low-density regions when finding center points.

D13 contains some adjacent sub-clusters and a U-shaped sub-cluster. DPC, DPC-KNN-PCA, SDPC, and FSDPC simultaneously select two high-density points on the U-shaped sub-cluster as the centers, and the corresponding centers of some low-density sub-clusters cannot be identified effectively. Similar to the circledata dataset, PS-DPC also finds samples from low-density areas as centers, which ignores high-density sub-clusters. The clustering results of DPCSA, LDP-MST, HC-LCCV, and KS-FDPC on D13 are superior to other algorithms, but similar to Pathbased, the selection of the centers of DPCSA relies heavily on human factors, and DPCSA does not provide effective strategies for the selection of the centers.

Table 5 shows the experimental results of the control algorithms on all synthesized datasets, where Par represents the parameter settings of each algorithm and the optimal results for each dataset are shown in bold. In terms of results, KS-FDPC achieves the best results on other datasets except for Pathbased. In the Pathbased, KS-FDPC is second only to DPCSA, which is caused by the misclassification of very few boundary points.

The above synthetic data is used to verify the improvement of KS-FDPC on the “Domino” effect of DPC, but the sample size is not high (<1000), which cannot verify the operation rate of it. For this purpose, the large-capacity composite datasets given in [30] are

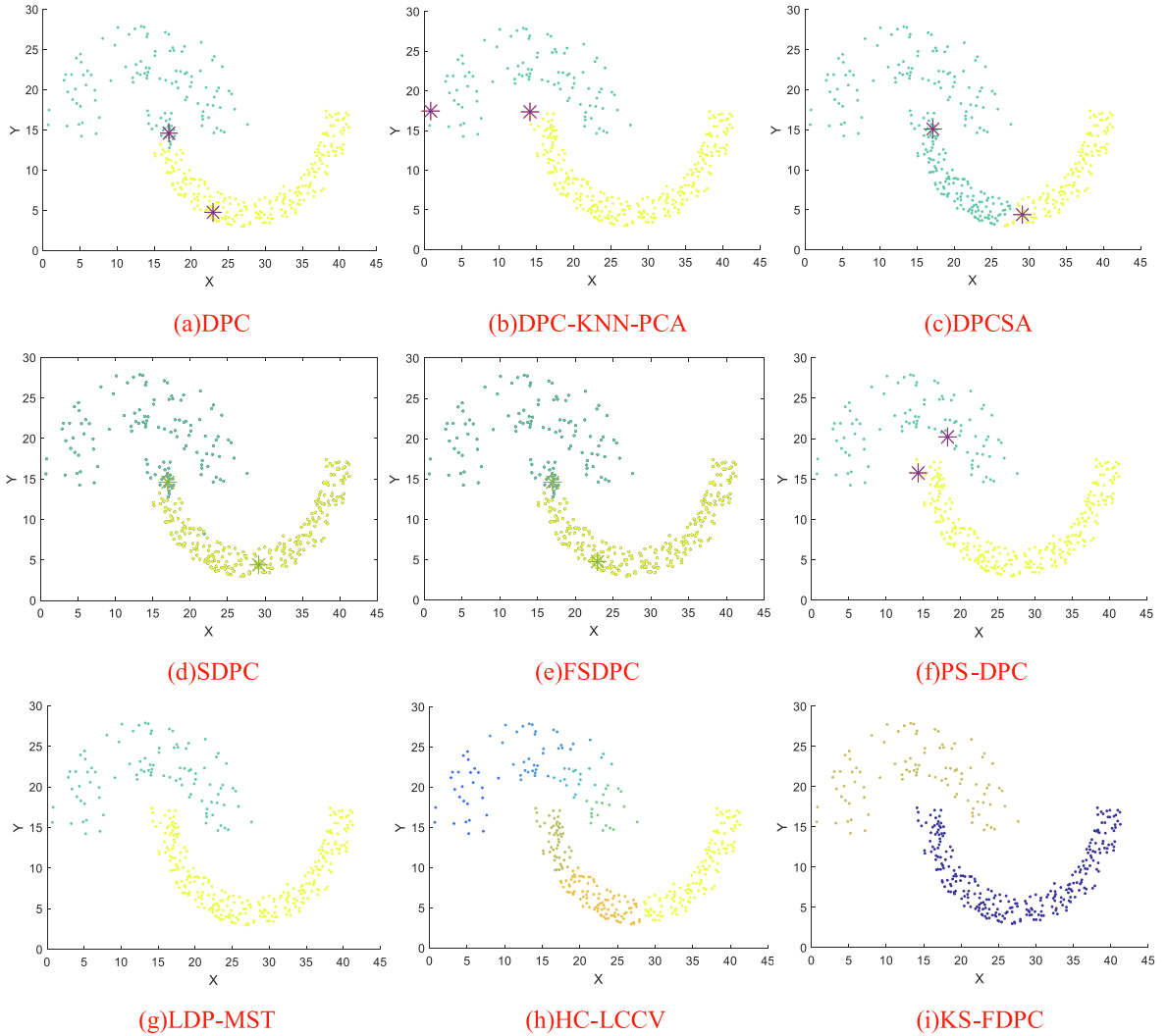


Fig. 6. Clustering results on the Jain dataset.

adopted, and the specific sampling density functions are shown in Fig. 9. Fig. 9 shows the raw data distribution of five datasets, with a sample size of 2^{17} for each dataset. Since most of the comparison algorithms mentioned above need to calculate the distance matrix, they cannot process the dataset with a capacity of 131,072 ($=2^{17}$, which is within the range of 2^{14} to 2^{24}). For this, only the LDP-MST algorithm is used as a comparison. For KS-FDPC and LDP-MST, 2^{17} is the most suitable data volume for comparison. The clustering results are shown in Fig. 10.

It can be seen from Fig. 10 that LDP-MST and KS-FDPC have similar clustering accuracy on the five large-capacity composite datasets, and all evaluation indicators are above 0.95. However, on these five datasets, the time consumption of KS-FDPC is lower than that of LDP-MST and is not easily affected by datasets of different shapes. The reason is that on the five datasets, the K value of the KS-FDPC algorithm is uniformly set to 200, so the amount of sub-clusters that need to be incorporated is small. While LDP-MST uses natural nearest neighbors to automatically determine the K value. Although the algorithm avoids the selection of parameters, the final determined K values are very small, resulting in a large number of sub-clusters being merged. From experiments, it can be known that the K value of LDP-MST on Flower and SF are 16 and 9 respectively, and the amount of initialized sub-clusters are 3563 and 6919 respectively. The final running time difference of LDP-MST on Flower and SF is more than 3000 s.

Moreover, experiments on large-capacity datasets show that KS-FDPC uses a sparse matrix to save storage space and has good results. LDP-MST also only saves the KNN of each point, which can also reduce the space resources of the algorithm. By comparing LDP-MST and KS-FDPC, it can be seen that KS-FDPC needs to store larger neighbors and fewer sub-cluster similarities, while LDP-MST needs to store fewer neighbors and more sub-cluster similarities. DPC, DPCSA, SDPC, FSDPC, DPC-KNN-PCA, HC-LCCV, and PS-DPC are limited by storage space and cannot run on large data.

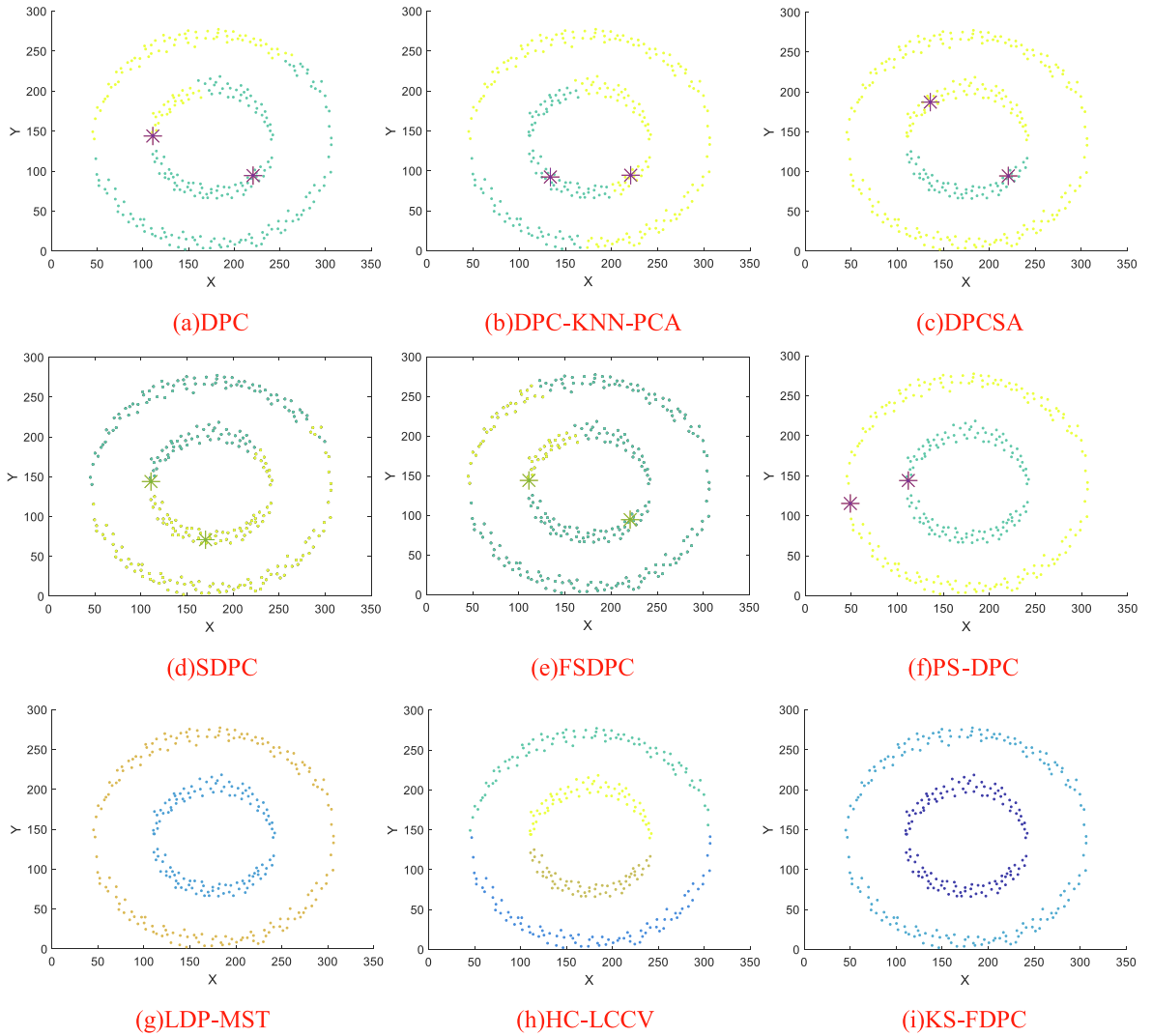


Fig. 7. Clustering results on the circledata dataset.

4.2. Results and analysis on UCI datasets

To further verify the effectiveness of KS-FDPC, we also conducted further experiments on eight UCI datasets. KS-FDPC algorithm will normalize the UCI datasets before processing them.

Table 6 shows the experimental results on UCI datasets. On the Letter dataset, compared to other algorithms, HC-LCCV does not produce clustering results due to its need for more memory resources. Among them, KS-FDPC obtains the optimal results on Wine, Seeds, Pendigits, and Letter. In Iris, the clustering performance of KS-FDPC is second only to LDP-MST, and the difference between different indicators of the two algorithms is within 2%. In Segmentation and Gamma, KS-FDPC's evaluation indicators can be ranked in the top three, even if they cannot reach the best. Hence, the sub-cluster partitioning and two-stage sub-cluster merging strategy of KS-FDPC improve DPC to some extent, for example, effectively eliminating the “Domino” effect of the DPC algorithm.

Compared with the improvement of the clustering effect, the improvement in computational efficiency of the KS-FDPC algorithm is more obvious. Fig. 11 shows the running time of each algorithm on the UCI datasets. Among them, UCI datasets are arranged in order of sample size from small to large. Because HC-LCCV did not generate clustering results on Letter, its execution rates will no longer be compared. As data capacity increases, the growth trend of the running time of the DPCSA algorithm is significantly more than that of other algorithms. FSDPC reduces the distance calculation of non-neighbor points, while SDPC filters out some sample points. Therefore, the running time of FSDPC and SDPC is lower than that of DPC, DPC-KNN-PCA, and PS-DPC. However, the running time of FSDPC and SDPC also has a certain growth trend with the increase of the datasets. For LDP-MST and KS-FDPC, their running time is much less than those of other algorithms, and they have little change with the increase in sample size. On the Letter dataset, LDP-MST runs slightly longer than KS-FDPC. Therefore, the running speed of KS-FDPC is superior to other control algorithms.

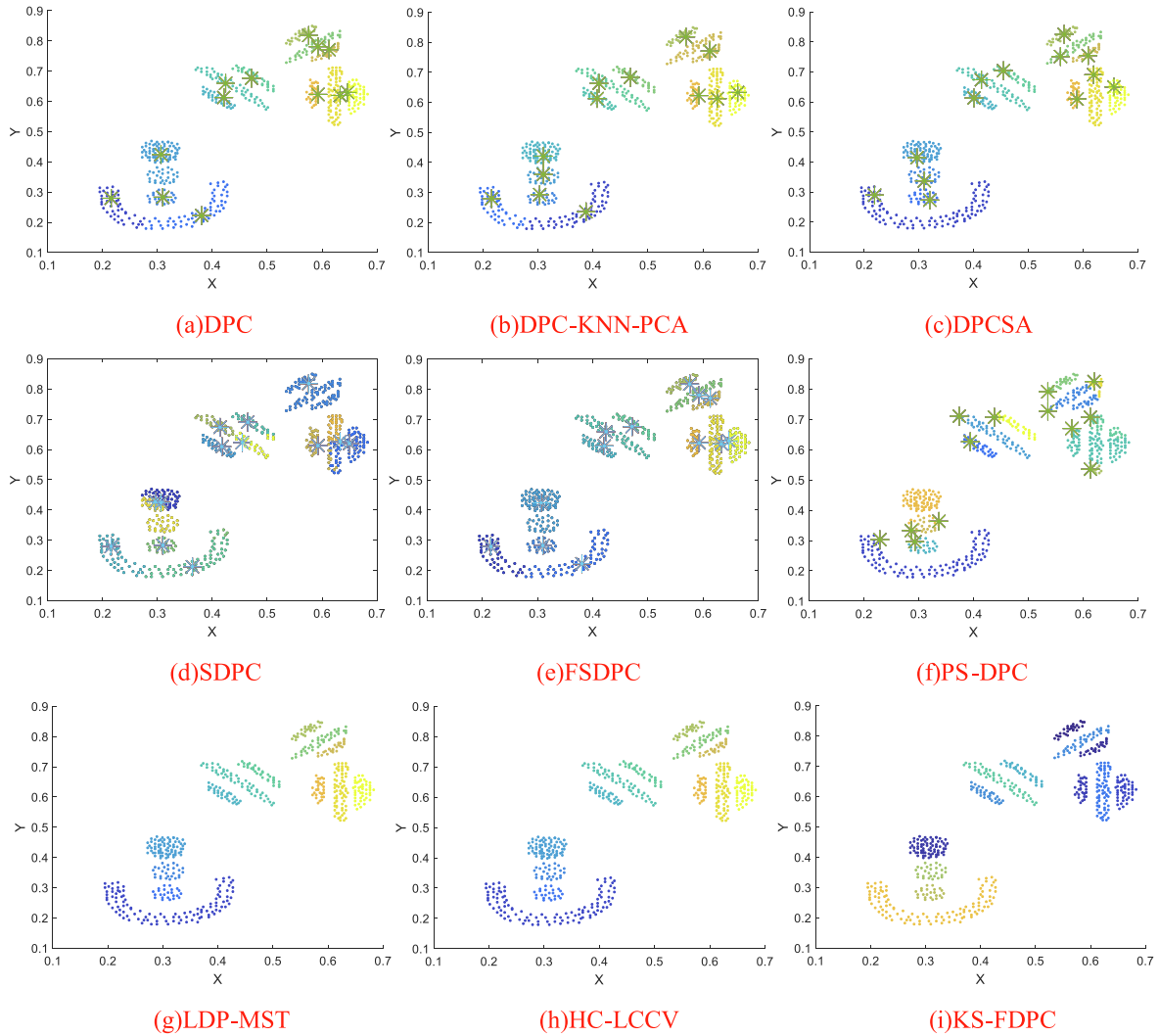


Fig. 8. Clustering results on the D13 dataset.

4.3. Ablation experiments

From the algorithm flow and analysis in Section 3.4, it can be seen that KS-FDPC has a higher rate and lower spatial complexity compared to DPC, which corresponds to contributions 1 and 3 in Introduction, respectively. In this section, we conduct ablation experiments to verify the importance of contributions 1 and 3.

Table 7 shows the experimental results of KS-FDPC on the CC dataset (Section 4.1, Fig. 9). $CC-2^n$ represents a CC dataset with a sampling capacity of 2^n . KS-FDPC-sparse means replacing the fully connected matrix of KS-FDPC with a sparse matrix. Table 7 shows that by using a sparse matrix, the KS-FDPC algorithm can process datasets with a capacity of 2^{18} or less. As a result, the amount of data that KS-FDPC can process has increased by 15 times.

Table 8 shows the comparison of runtime when replacing Kd-tree with brute force nearest-neighbor search in KS-FDPC. KS-FDPC-Full and KS-FDPC correspond to algorithms that use brute force nearest-neighbor search and Kd-tree nearest-neighbor search, respectively. Because KS-FDPC-Full removes duplicate sample points during the preprocessing, the running time on Letter is less than that on Gamma, and the higher dimensional features of Letter increase the time of the Kd-tree nearest-neighbor search. As the sample size increases, the ratio of the running time between KS-FDPC-Full and KS-FDPC also increases. On Gamma and Letter, KS-FDPC is about 47 and 29 times faster than KS-FDPC-Full respectively, which is better than a decrease in space complexity because KS-FDPC-sparse cannot run on datasets with a capacity of 2^{19} or more.

It is worth noting that KS-FDPC uses a sparse matrix to store data with a capacity of more than 20,000, which will lead to a rapid increase in running time. For example, by comparing Fig. 10 and Fig. 11, it can be seen that the running time of KS-FDPC on Letter is within 30 s, while the running time on $CC-2^{17}$ is close to 1000 s. This means that KS-FDPC can run quickly on datasets with a capacity of up to 20000, while on larger datasets, KS-FDPC can run with low efficiency. Therefore, rapid operation on large-scale data is one of the

Table 5
Clustering results on eight synthetic datasets.

Algorithm	ACC	ARI	AMI	Par	ACC	ARI	AMI	Par
Pathbased					Compound			
DPC	0.7333	0.4530	0.5360	2%	0.8722	0.6346	0.8327	1%
DPC-KNN-PCA	0.7800	0.5067	0.5794	2	0.8797	0.8087	0.8465	0.6
DPCSA	1.0000	1.0000	1.0000	5	0.8396	0.8284	0.8392	5
SDPC	0.7433	0.4618	0.5433	2%/0.7	0.8321	0.5752	0.7741	1%/0.7
FSDPC	0.7333	0.4530	0.5360	2%	0.8722	0.6346	0.8327	1%
PS-DPC	0.9500	0.8593	0.8488	1%/6.6	0.8897	0.8169	0.8427	1%/3.3
LDP-MST	0.6733	0.4113	0.4956	–	0.8271	0.8338	0.8485	–
HC-LCCV	0.6300	0.3899	0.4679	–	0.8722	0.8531	0.9105	–
KS-FDPC	0.9933	0.9794	0.9697	12	0.9975	0.9972	0.9923	8
Aggregation					Spiral			
DPC	1.0000	1.0000	1.0000	4%	1.0000	1.0000	1.0000	2%
DPC-KNN-PCA	0.9975	0.9956	0.9923	3	1.0000	1.0000	1.0000	2
DPCSA	0.9734	0.9581	0.9537	5	1.0000	1.0000	1.0000	5
SDPC	0.9962	0.9920	0.9877	0.1%/0.7	0.6987	0.2887	0.3772	2%/0.7
FSDPC	0.9949	0.9913	0.9868	0.1%	1.0000	1.0000	1.0000	2%
PS-DPC	1.0000	1.0000	1.0000	1%/1.8	1.0000	1.0000	1.0000	1%/1.1
LDP-MST	0.9975	0.9956	0.9923	–	1.0000	1.0000	1.0000	–
HC-LCCV	0.8274	0.8089	0.8882	–	1.0000	1.0000	1.0000	–
KS-FDPC	1.0000	1.0000	1.0000	32	1.0000	1.0000	1.0000	12
flame					Jain			
DPC	1.0000	1.0000	1.0000	6%	0.9223	0.7055	0.6439	1%
DPC-KNN-PCA	1.0000	1.0000	1.0000	0.5	1.0000	1.0000	1.0000	0.1
DPCSA	1.0000	1.0000	1.0000	5	0.6247	0.0442	0.2167	5
SDPC	0.9333	0.7497	0.6779	6%/0.7	0.9196	0.6965	0.6365	1%/0.7
FSDPC	1.0000	1.0000	1.0000	6%	0.9223	0.7055	0.6439	1%
PS-DPC	1.0000	1.0000	1.0000	1%/0.1	1.0000	1.0000	1.0000	1%/3.6
LDP-MST	0.9833	0.9339	0.8748	–	1.0000	1.0000	1.0000	–
HC-LCCV	0.9917	0.5827	0.6611	–	1.0000	0.3022	0.5024	–
KS-FDPC	1.0000	1.0000	1.0000	16	1.0000	1.0000	1.0000	9
circledata					D13			
DPC	0.5767	0.0088	0.0242	0.1%	0.9541	0.8365	0.9516	4%
DPC-KNN-PCA	0.6767	0.1207	0.0786	1.4	0.9575	0.8639	0.9542	5
DPCSA	0.7833	0.3141	0.3156	5	1.0000	1.0000	1.0000	5
SDPC	0.5767	–0.0024	–0.0006	0.1%/0.7	0.7942	0.5990	0.8342	4%/0.7
FSDPC	0.5767	0.0128	0.0020	0.1%	0.9541	0.8365	0.9516	4%
PS-DPC	1.0000	1.0000	1.0000	1%/0.1	0.8095	0.7632	0.8660	1%/0.1
LDP-MST	1.0000	1.0000	1.0000	–	1.0000	1.0000	1.0000	–
HC-LCCV	1.0000	0.4917	0.6612	–	1.0000	1.0000	1.0000	–
KS-FDPC	1.0000	1.0000	1.0000	9	1.0000	1.0000	1.0000	6

‘–’ indicates that no input parameters are required.

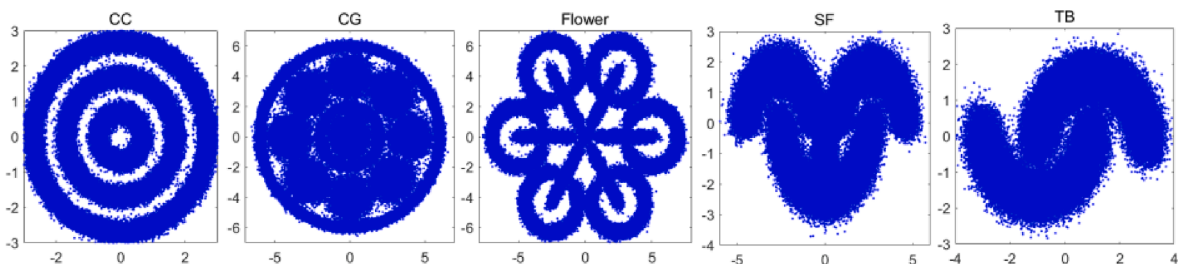


Fig. 9. Five density functions for sampling.

problems that will be solved in the next.

4.4. Comparison of KS-FDPC and IMKNN

In Sections 2.2 and 3.1, we mentioned that when searching for IMKNN of sample points, the search range is reduced to 2 K-nearest neighbors. The original author used a global search [22], that is, each Euclidean distance in the distance matrix is compared with its corresponding two distance levels. To demonstrate the effectiveness of the nearest-neighbor search in KS-FDPC, we compare the clustering results of KS-FDPC with $M_{k-NN}G$ -DPC. The results and experimental data of $M_{k-NN}G$ -DPC are from the original literature [22], and the evaluation indicators are ACC and ARI. Except for Iris, Seeds, Segmentation, and Waveform in Table 4, the remaining datasets

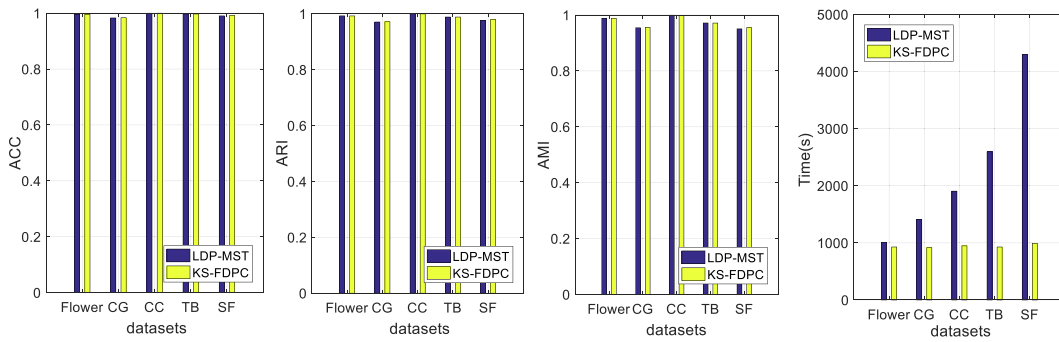


Fig. 10. Comparison results of LDP-MST and KS-FDPC.

Table 6

Clustering results on eight UCI datasets.

Algorithm	ACC	ARI	AMI	Par	ACC	ARI	AMI	Par
Iris					Wine			
DPC	0.9400	0.8343	0.8222	0.2%	0.7191	0.4070	0.3848	0.5%
DPC-KNN-PCA	0.8000	0.5762	0.6287	4	0.8989	0.7191	0.7320	3
DPCSA	0.9667	0.9038	0.8831	5	0.9101	0.7414	0.7480	5
SDPC	0.8667	0.6828	0.7402	0.2%/0.7	0.7303	0.4311	0.4073	0.5%/0.7
FSDPC	0.9400	0.8343	0.8222	0.2%	0.7191	0.4070	0.3848	0.5%
PS-DPC	0.9400	0.8343	0.8222	1%/0.1	0.7079	0.3715	0.4131	1%/4.6
LDP-MST	0.9733	0.9222	0.8999	–	0.7135	0.3627	0.4267	–
HC-LCCV	0.6667	0.5681	0.7316	–	0.7135	0.2723	0.3834	–
KS-FDPC	0.9667	0.9038	0.8836	19	0.9157	0.7562	0.7600	11
Seeds					Segmentation			
DPC	0.8952	0.7170	0.6715	1%	0.6442	0.4081	0.5870	1%
DPC-KNN-PCA	0.8952	0.7144	0.6573	2	0.6926	0.5671	0.6801	6
DPCSA	0.8810	0.6873	0.6609	5	0.7199	0.5429	0.6623	5
SDPC	0.8620	0.6493	0.6571	1%/0.7	0.4939	0.2747	0.4029	1%/0.3
FSDPC	0.8952	0.7170	0.6715	1%	0.6372	0.4037	0.5774	1%
PS-DPC	0.8952	0.7170	0.6715	1%/0.4	0.2926	0.1096	0.3189	1%/0.9
LDP-MST	0.8191	0.5702	0.6274	–	0.6147	0.3760	0.6855	–
HC-LCCV	0.5619	0.2506	0.3999	–	0.2857	0.1023	0.3474	–
KS-FDPC	0.9048	0.7448	0.7284	32	0.6909	0.5632	0.6876	120
Waveform					Pendigits			
DPC	0.5832	0.2960	0.3528	0.5%	0.7205	0.5655	0.7350	2%
DPC-KNN-PCA	0.6482	0.2567	0.3041	0.1	0.7699	0.6340	0.7647	0.2
DPCSA	0.6284	0.2334	0.2604	5	0.7382	0.6586	0.7654	5
SDPC	0.5934	0.2804	0.3243	0.5%/0.3	0.6903	0.5064	0.6513	2%/0.3
FSDPC	0.5890	0.2751	0.3193	0.5%	0.7612	0.6462	0.7466	2%
PS-DPC	0.6064	0.3232	0.3770	1%/1.1	0.7680	0.6519	0.7568	1%/7.6
LDP-MST	0.6610	0.2931	0.3495	–	0.7809	0.5984	0.7696	–
HC-LCCV	0.5210	0.1535	0.2105	–	0.5415	0.3459	0.6505	–
KS-FDPC	0.6618	0.2735	0.3280	60	0.8857	0.7838	0.8563	95
Gamma					Letter			
DPC	0.6484	–0.0009	0.0000	1%	0.3288	0.1721	0.4460	0.05%
DPC-KNN-PCA	0.6484	–0.0016	0.0000	1	0.3590	0.1328	0.4717	0.7
DPCSA	0.6560	0.0131	0.0471	5	0.2226	0.0374	0.4218	5
SDPC	0.6484	0.0393	0.0108	1%/0.3	0.2907	0.0932	0.3828	0.05%/0.3
FSDPC	0.6484	–0.0030	0.0000	1%	0.3291	0.1732	0.4471	0.05%
PS-DPC	0.6493	0.0015	0.0028	1%/5.6	0.2778	0.0561	0.3730	1%/0.6
LDP-MST	0.6484	–0.0040	0.0001	–	0.3759	0.1287	0.4565	–
HC-LCCV	0.6857	0.0659	0.0425	–	–	–	–	–
KS-FDPC	0.6646	0.0331	0.0288	190	0.3904	0.1735	0.4965	176

‘–’ indicates that no input parameters are required or there is no output result.

used are shown in Table 9.

From Fig. 12 and Fig. 13, it can be seen that KS-FDPC has indeed caused a certain degree of decrease in accuracy due to narrowing the search range. On Movement, Waveform, Segmentation, and Ecoil, the ARI values of KS-FDPC are lower than those of $M_{k-NN}G-DPC$, while on Ionosphere, the clustering results of KS-FDPC are better than those of $M_{k-NN}G-DPC$. We believe that the loss of accuracy is within an acceptable range, and the resulting efficiency improvement is also one of the advantages of KS-FDPC.

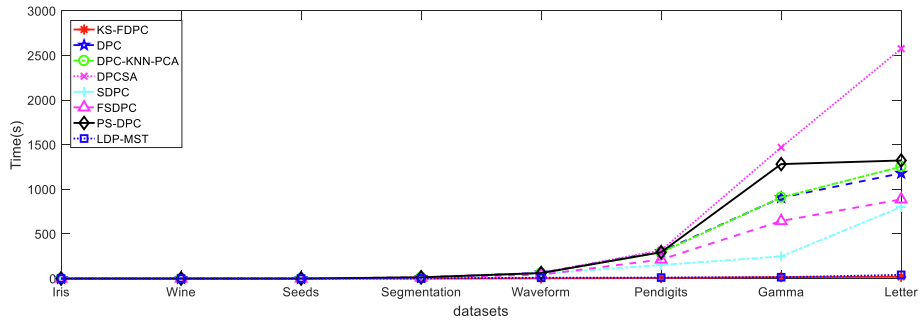


Fig. 11. Comparison of running time on UCI datasets.

Table 7

Clustering results of KS-FDPC on CC with different sizes.

Algorithm		CC-2 ¹⁴	CC-2 ¹⁵	CC-2 ¹⁶	CC-2 ¹⁷	CC-2 ¹⁸	CC-2 ¹⁹
KS-FDPC	ACC	0.9996	–	–	–	–	–
	ARI	0.9990	–	–	–	–	–
	AMI	0.9966	–	–	–	–	–
KS-FDPC-sparse	ACC	0.9996	0.9999	0.9998	0.9999	0.9999	–
	ARI	0.9990	0.9999	0.9995	0.9996	0.9997	–
	AMI	0.9966	0.9994	0.9983	0.9987	0.9989	–

‘–’ indicates that there is no output result.

Table 8

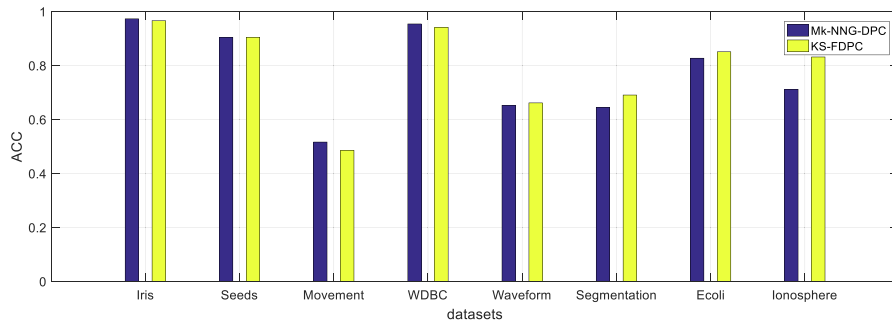
Running time of KS-FDPC-Full and KS-FDPC on eight UCI datasets.

Algorithm	Iris	Wine	Seeds	Segmentation	Waveform	Pendigits	Gamma	Letter
KS-FDPC-Full	0.0250	0.0297	0.0300	1.0220	4.1603	17.9535	756.6904	574.4331
KS-FDPC	0.0208	0.0264	0.0213	0.7590	2.1710	6.7714	16.1794	19.6335

Table 9

UCI datasets for comparison between KS-FDPC and $M_{k-NN}G$ -DPC.

Datasets	Sizes	Attributes	Clusters
Movement	360	90	15
WDBC	569	30	2
Ecoli	336	7	8
Ionosphere	351	34	2

Fig. 12. ACC of $M_{k-NN}G$ -DPC and KS-FDPC.

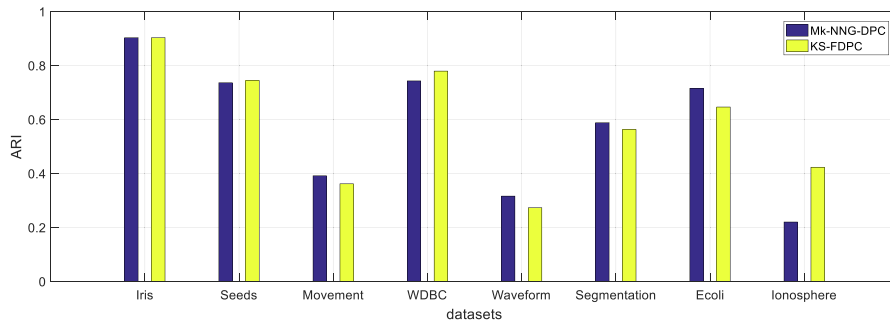


Fig. 13. ARI of $M_{k-NN}G$ -DPC and KS-FDPC.

4.5. Parameter analysis

Since the parameter K has a significant impact on the results of KS-FDPC, we need to analyze the K value. During the operation of KS-FDPC, users need to manually set the parameter K . Meanwhile, the connectivity between samples in the IMKNN method is weaker than that in other KNN methods, so too small K will cause a large number of sporadic sub-clusters. To avoid this, it is necessary to set the K value as large as possible, which makes it difficult to choose the parameter. Fig. 14 shows the clustering accuracy generated by different K values of KS-FDPC on the five selected UCI datasets. The K value for Iris and Wine is in the range [6, 20] with a step size of 1. Segmentation, Waveform, and Pendigits have K values in the range [50, 120] with a step size of 5. The abscissa represents different ranges of K values.

From Fig. 14, we can know that KS-FDPC is not sensitive to the change of K value on Segmentation and Waveform. On Iris, Wine, and Pendigits, the ACC of KS-FDPC has obvious fluctuations with respect to the change of K value. Meanwhile, on the five datasets, the clustering accuracy has a certain upward trend with the increase of the K value. Therefore, KS-FDPC tends to select large K values, which will bring difficulties for parameter selection.

Through the experiment in this section, we can conclude that KS-FDPC can available cut down the execution time of DPC, and greatly alleviate the impact of the “Domino” effect in DPC, so KS-FDPC is an effective improvement of the DPC algorithm. However, the KS-FDPC has a slow processing speed on large-scale data (Section 4.3) and it is easily affected by the value of parameter K , which are aspects that need to be improved.

5. Conclusion

In this article, a fast density peaks clustering algorithm based on improved mutual K -nearest neighbors and sub-cluster merging is proposed. This algorithm uses Kd-tree to accelerate the nearest neighbors search while dividing the dataset into multiple sub-clusters to alleviate the “Domino” effect. To deal with large-scale datasets, the fully connected matrix of the algorithm is replaced by a sparse matrix to reduce the space complexity.

In the experiment, we mainly compared KS-FDPC with eight improved DPC algorithms on eight synthetic datasets and eight UCI datasets. The results show that KS-FDPC is superior to all control algorithms and can significantly reduce the running time of the algorithm while maintaining clustering accuracy. By comparing KS-FDPC and $M_{k-NN}G$ -DPC, it can be seen that implementing neighbor search on improved mutual K -nearest neighbors can still maintain clustering effectiveness to a certain extent. Therefore, KS-FDPC can serve as an improved algorithm of DPC.

However, the KS-FDPC algorithm is relatively sensitive to parameters and its running speed on large capacity datasets is not ideal.

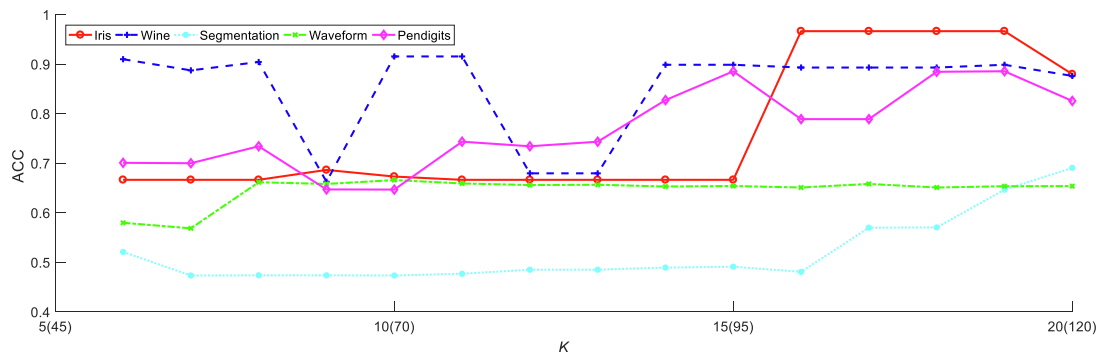


Fig. 14. Influence of K value on clustering results.

Therefore, in the following work, we will further adjust the improved mutual K-nearest neighbors to increase the robustness of the algorithm. At the same time, it is necessary to improve the calculation of similarity between sub-clusters in KS-FDPC to avoid storing the distance matrix. Finally, replacing Kd-tree with a more effective approximate nearest neighbors search to improve the algorithm speed.

CRedit authorship contribution statement

Chao Li: Conceptualization, Methodology, Software. **Shifei Ding:** Supervision. **Xiao Xu:** Supervision. **Haiwei Hou:** Writing – review & editing. **Ling Ding:** .

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgement

This work is supported by the National Natural Science Foundation of China (no.62276265, no.61976216 and no.62206296).

References

- [1] D. Fuentealba, M. López, H. Ponce, Effects on time and quality of short text clustering during real-time presentations, *IEEE Lat. Am. Trans.* 19 (8) (2021) 1391–1399.
- [2] Rudrappa G, Vijapur N. Cloud Classification using K-Means Clustering and Content based Image Retrieval Technique. 2020 International Conference on Communication and Signal Processing (ICCCSP), Chennai, India, 2020: 0700-0704.
- [3] A. Pister, P. Buono, J.-D. Fekete, C. Plaisant, P. Valdivia, Integrating prior knowledge in mixed-initiative social network clustering, *IEEE Trans. Vis. Comput. Graph.* 27 (2) (2021) 1775–1785.
- [4] Peng M, Hu B, Tan Z. Hierarchical Structure of E-commerce Big Data Clustering Based on Hadoop Platform. 2021 5th International Conference on Electronics, Communication and Aerospace Technology (ICECA), Coimbatore, India, 2021: 1376–1379.
- [5] M. Li, D. Xu, D. Zhang, J. Zou, The seeding algorithms for spherical k-means clustering, *J. Glob. Optim.* 76 (4) (2020) 695–708.
- [6] L. Yao, Z. Ge, Scalable semisupervised GMM for big data quality prediction in multimode processes, *IEEE Trans. Ind. Electron.* 66 (5) (2019) 3681–3692.
- [7] E. Zhu, Y. Zhang, P. Wen, F. Liu, Fast and stable clustering analysis based on Grid-mapping K-means algorithm and new clustering validity index, *Neurocomputing* 363 (2019) 149–170.
- [8] Jebari S, Smiti A, Louati A. AF-DBSCAN: An unsupervised Automatic Fuzzy Clustering method based on DBSCAN approach. 2019 IEEE International Work Conference on Bioinspired Intelligence (IWOB), Budapest, Hungary, 2019: 1–6.
- [9] Xu X, Ding S, Sun T. A Fast Density Peaks Clustering Algorithm Based on Pre-Screening. 2018 IEEE International Conference on Big Data and Smart Computing (BigComp), Shanghai, China, 2018: 513–516.
- [10] Macqueen J. Some methods for classification and analysis of multivariate observations. *Proceedings of the 5th Berkeley symposium on mathematical statistics and probability*, San Francisco, USA, 1967: 281–297.
- [11] A.P. Dempster, N.M. Laird, D.B. Rubin, Maximum likelihood from incomplete data via the EM algorithm, *J. R. Stat. Soc. Ser. B* 39 (1) (1977) 1–22.
- [12] S. Ding, C. Li, X. Xu, L. Ding, J. Zhang, L. Guo, T. Shi, A sampling-based density peaks clustering algorithm for large-scale data, *Pattern Recogn.* 136 (2023).
- [13] T. Ouyang, W. Pedrycz, N.J. Pizzi, Rule-based modeling with DBSCAN-based information granules, *IEEE Trans. Cybern.* 51 (7) (2021) 3653–3663.
- [14] A. Rodriguez, A. Laio, Clustering by fast search and find of density peaks, *Science* 344 (6191) (2014) 1492–1496.
- [15] X. Xu, S. Ding, M. Du, Y.-u. Xue, DPCG: an efficient density peaks clustering algorithm based on grid, *Int. J. Mach. Learn. Cybern.* 9 (5) (2018) 743–754.
- [16] J. Zhao, J. Tang, T. Fan, C. Li, L. Xu, Density peaks clustering based on circular partition and grid similarity, *Concurr Comput: Practice Exp* 32 (7) (2020).
- [17] R. Liu, H. Wang, X. Yu, Shared-nearest-neighbor-based clustering by fast search and find of density peaks, *Inf. Sci.* 450 (2018) 200–226.
- [18] Q.i. Diao, Y. Dai, Q. An, W. Li, X. Feng, F. Pan, Clustering by detecting density peaks and assigning points by similarity-first search based on weighted K-nearest neighbors graph, *Complexity* 2020 (2020) 1–17.
- [19] J. Xie, H. Gao, W. Xie, X. Liu, P.W. Grant, Robust clustering by detecting density peaks and assigning points based on fuzzy weighted K-nearest neighbors, *Inf. Sci.* 354 (2016) 19–40.
- [20] S. Ding, W. Du, X. Xu, T. Shi, Y. Wang, C. Li, An improved density peaks clustering algorithm based on natural neighbor with a merging strategy, *Inf. Sci.* 624 (2023) 252–276.
- [21] J. Chen, P.S. Yu, A domain adaptive density clustering algorithm for data with varying density distribution, *IEEE Trans. Knowl. Data Eng.* 33 (6) (2021) 2310–2321.
- [22] J. Fan, P. Jia, L. Ge, $M_{k-NN}G$ -DPC: density peaks clustering based on improved mutual K-nearest-neighbor graph, *Int. J. Mach. Learn. Cybern.* 11 (2020) 1179–1195.
- [23] M. Du, S. Ding, H. Jia, Study on density peaks clustering based on k-nearest neighbors and principal component analysis, *Knowl.-Based Syst.* 99 (2016) 135–145.
- [24] D. Yu, G. Liu, M. Guo, X. Liu, S. Yao, Density peaks clustering based on weighted local density sequence and nearest neighbor assignment, *IEEE Access* 7 (2019) 34301–34317.
- [25] X. Xu, S. Ding, Z. Shi, An improved density peaks clustering algorithm with fast finding cluster centers, *Knowl.-Based Syst.* 158 (2018) 65–74.
- [26] X. Xu, S. Ding, Y. Wang, L. Wang, W. Jia, A fast density peaks clustering algorithm with sparse search, *Inf. Sci.* 554 (2021) 61–83.
- [27] Z. Bian, F.L. Chung, S. Wang, Fuzzy density peaks clustering, *IEEE Trans. Fuzzy Syst.* 29 (7) (2021) 1725–1738.
- [28] L. Sun, S. Bao, S. Ci, X. Zheng, L. Guo, Y. Luo, Differential privacy-preserving density peaks clustering based on shared near neighbors similarity, *IEEE Access* 7 (2019) 89427–89440.
- [29] N.X. Vinh, J. Epps, J.C. Bailey, Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance, *J. Mach. Learn. Res.* 11 (2010) 2837–2854.
- [30] T. Qiu, Y. Li, Fast LDP-MST: an efficient density-peak-based clustering method for large-size datasets, *IEEE Trans. Knowl. Data Eng.* (2022).

- [31] D. Cheng, Q. Zhu, J. Huang, Q. Wu, L. Yang, Clustering with local density peaks-based minimum spanning tree, *IEEE Trans. Knowl. Data Eng.* 33 (2) (2021) 374–387.
- [32] B. Wu, B.M. Wilamowski, A fast density and grid based clustering method for data with arbitrary shapes and noise, *IEEE Trans. Ind. Inf.* 13 (4) (2017) 1620–1628.
- [33] S. Sieranoja, P. Fränti, Fast and general density peaks clustering, *Pattern Recogn. Lett.* 128 (2019) 551–558.
- [34] J. Lu, Y. Zhao, K.-L. Tan, Z. Wang, Distributed density peaks clustering revisited, *IEEE Trans. Knowl. Data Eng.* 34 (8) (2022) 3714–3726.
- [35] Y. Wang, D. Wang, Y. Zhou, et al., VDPC: variational density peaks clustering algorithm, *Inf. Sci.* 621 (2023) 627–651.
- [36] Y. Wang, D. Wang, X. Zhang, W. Pang, C. Miao, A.-H. Tan, Y. Zhou, McDPC: multi-center density peak clustering, *Neural Comput. & Applic.* 32 (17) (2020) 13465–13478.
- [37] C. Wu, J. Lee, T. Isokawa, J. Yao, Y. Xia, Efficient clustering method based on density peaks with symmetric neighborhood relationship, *IEEE Access* 7 (2019) 60684–60696.
- [38] D. Cheng, Q. Zhu, J. Huang, Q. Wu, L. Yang, A novel cluster validity index based on local cores, *IEEE Trans. Neural Networks Learn. Syst.* 30 (4) (2019) 985–999.
- [39] Tobin J, Zhang M. DCF: An Efficient and Robust Density-Based Clustering Method. 2021 IEEE International Conference on Data Mining (ICDM), Auckland, New Zealand, 2021: 629–638.